

DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

Anonymous authors
Anonymous submission

ABSTRACT

Parametric imitation learning through behavior cloning is broadly useful but can generalize poorly to states encountered off the demonstration distribution, where compounding deployment errors expose regions that were not directly supervised. We study whether the same demonstration corpus used for training can also support inference through a semi-parametric, retrieval-based policy. We present **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, which reframes imitation learning around local neighborhood structure rather than only direct state-to-action prediction. For each query, DARP conditions action prediction on k -nearest expert neighbors, their actions, and relative difference vectors between neighbor states and the query state. This design stays within the usual behavior-cloning information regime: it does not add data collection, online expert feedback, or task-specific knowledge. Across the evaluated continuous-control and robotic-manipulation benchmarks, including low-dimensional states and high-dimensional visual features, DARP improves over standard behavior cloning by 15-46%. Code and demos will be released upon acceptance.

1 INTRODUCTION

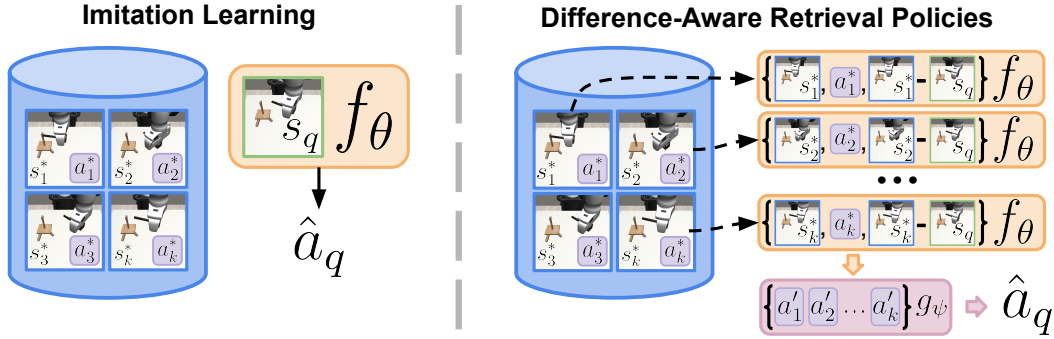


Figure 1: **Overview of DARP:** Standard BC (left) fits a parametric mapping and discards the expert dataset at inference time, whereas DARP (right) keeps that dataset available as a local reference set. For each query point s_q , DARP retrieves nearby expert state-action pairs and predicts through a difference-aware reparameterization built from query-neighbor displacement vectors.

Behavior cloning (BC) (Pomerleau, 1991) is a widely used route to imitation learning because it converts expert demonstrations into an ordinary supervised learning problem, enabling robotic policies for dexterous and long-horizon behaviors (Zhao et al., 2023; Chi et al., 2024; Black et al., 2024; Chung et al., 2014). The same simplicity also limits deployment robustness across sequential decision-making settings (Ross et al., 2011): small prediction errors can compound over a rollout and move the agent into states that are weakly represented, or absent, in the demonstrations (Ross et al., 2011; Spencer et al., 2021). In these out-of-distribution regions, a policy trained only by pointwise supervision can produce unstable, high-variance actions that derail the task.

Many methods address compounding error by expanding the information available to the learner (Ross et al., 2011; Venkatraman et al., 2015; Ke et al., 2024b; Levine et al., 2020). Such approaches can

be effective, but they often assume simulators, interactive experts, large amounts of sub-optimal experience, or task-specific structure. We instead ask what can be gained without leaving the standard BC data regime: the learner receives only the original expert state-action pairs and no additional supervision or feedback. This leads to our central question: *can the demonstration dataset itself be reused at inference time to reduce the variance of BC policies?*

Statistically, BC controls supervised risk on expert states but does not directly constrain how predictions vary between or beyond those states. As a result, overparameterized policies may oscillate in low-density regions of the state space, precisely the regions closed-loop rollouts can encounter after small mistakes. A broadly applicable remedy is to encourage *smoothness*: nearby states should induce compatible actions when the expert data support such consistency (Kobayashi, 2022; Asadi et al., 2018; Ke et al., 2024a; Chen et al., 2024). Smoothness discourages spurious local fluctuations and can improve rollout stability; Section 4 discusses related strategies for imposing this bias.

These strategies cover several useful regimes, but each leaves gaps for general imitation learning: augmentation need not produce consistent labels, global continuity priors can blur legitimately distinct behaviors, temporal penalties constrain only along trajectories rather than across the state space, and explicit graph penalties introduce additional smoothness hyperparameters.

A complementary perspective is the distinction between global and local prediction. “Global” supervised policies (Black et al., 2024; Zhao et al., 2023; Chi et al., 2024) compress the demonstration set into one parametric function, which can be fragile when the rollout distribution shifts. “Local” methods (Pari et al., 2022; Mansimov & Cho, 2018; Salzberg & Aha, 1994) keep the training distribution available and adapt each prediction using neighborhoods of similar examples, yielding non-parametric or semi-parametric behavior grounded in observed expert data. Locality can improve robustness, but it is not sufficient by itself: performance depends on the retrieval metric, naive neighbor averaging can mix incompatible actions and underrepresent multimodality, and using neighbors only as absolute states can limit how much structure transfers across nearby queries.

We introduce **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric policy class that combines local grounding with the stability benefits of regularized prediction. At inference time, DARP (Fig. 1) does not act from the query state alone. It retrieves k neighbors from the demonstration corpus, forms tuples containing each neighbor state, its action, and its difference from the query, and predicts neighbor-conditioned candidate actions. A permutation-invariant aggregation step then converts these candidates into the executed action. This design preserves the standard BC assumptions while using the training set as a reusable local reference: no additional data, online supervision, or task-specific knowledge is introduced. In spectral terms, the resulting aggregation approximates a Laplacian filter on the k -NN (k -nearest neighbor) graph of expert states, giving a parameter-free smoothing mechanism that adapts to the local density and geometry of the dataset.

We support this framing theoretically and empirically while keeping the study within the same datasets, tasks, and supervision available to BC. Our analysis connects DARP to Laplacian regularization and shows that it implicitly applies a fixed low-pass spectral filter that suppresses high-frequency variance. In the evaluated imitation-learning benchmarks, DARP yields 15–46% improvements over standard behavior cloning across continuous control (MuJoCo), robotic manipulation (Robosuite, Robocasa), and visual-feature manipulation tasks (Robosuite with image state). The experiments also show that the architecture can be instantiated with richer policy components, including transformers and Gaussian mixture models. Finally, ablations isolate the representation and aggregation choices that drive the gains, offering broader design guidance for retrieval-augmented policies in sequential decision-making.

2 DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

We develop a class of imitation-learning policies that combines the broad coverage of parametric learning with the data-adaptive behavior of local methods. The resulting architecture and training objective operate under the same supervision assumptions as standard behavior cloning, while providing theoretical and empirical improvements in the settings we evaluate. This section derives the main guarantees from first principles; readers focused on the practical algorithm may skip to Section 2.5 for a self-contained description of DARP and then proceed to Section 3.

We begin by defining the imitation-learning setting (Section 2.1) and then introduce a regularized variant (Section 2.2) that uses neighborhood structure in the data to improve variance, generalization, and stability. Section 2.3 shows that the same smoothing effect can be obtained through policy *architecture* rather than by changing the loss. Section 2.4 then presents the practical algorithm, DARP, as a semi-parametric retrieval-augmented architecture that can be instantiated with standard neural networks and richer generative policy heads.

2.1 PRELIMINARIES: BEHAVIOR CLONING FOR IMITATION LEARNING

We use the standard finite-horizon imitation-learning formulation as a Markov Decision Process (MDP), $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P_0\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and P_0 is the initial state distribution. A *policy* maps states to action distributions, $f_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$, for the task objective of interest; when discussing the function class generally, we omit the subscript θ . The learner receives an expert dataset $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ of state-action pairs, with s^* and a^* reserved for states and actions in that dataset. Behavior cloning (Pomerleau, 1991) learns f_θ by treating imitation as supervised learning, $\arg \max_\theta \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\log(f_\theta(a^* | s^*))]$. The policy distribution can in principle be a rich generative model (Lipman et al., 2023; Chi et al., 2024); for exposition, we begin with a Gaussian parameterization and relax this choice in Section 2.4.

2.2 WARM-UP: NEIGHBOR MANIFOLD REGULARIZED IMITATION LEARNING

Behavior cloning minimizes supervised imitation loss on expert states sampled from $\mathcal{D}^*$, but this pointwise objective does not determine how the policy should vary away from the expert-state manifold. During deployment, accumulated errors can therefore place the agent in regions where an overparameterized BC policy is weakly constrained and may act unpredictably (Ross & Bagnell, 2010).

A natural way to use more of the dataset structure is to constrain not only predictions at individual training states, but also relationships among nearby states and their actions. We therefore consider an objective that explicitly promotes *local consistency*: expert states that are close under the chosen neighborhood graph should receive compatible predictions. This yields the neighborhood-regularized loss in Eq. 1 ($\mathcal{L}_{\text{MRIL}}$), which augments the standard imitation objective ($\mathcal{L}_{\text{BC}}$) with a smoothness penalty ($\mathcal{L}_S$) so the learned policy respects the geometry of the demonstrations rather than relying solely on pointwise labels.

$$\mathcal{L}_{\text{MRIL}}(f) = \underbrace{\mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)]}_{\text{supervised risk}(\mathcal{L}_{\text{BC}})} + \underbrace{\lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right]}_{\text{smoothness regularizer}(\mathcal{L}_S)}, \quad (1)$$

Here $\ell(f(s^*), a^*)$ is the supervised imitation loss, $\mathcal{N}_k(s^*)$ denotes the k -nearest neighbors of s^* in the expert dataset, and $w_i(s^*)$ are normalized kernel weights based on state differences, $w_i(s^*) \propto K_\Delta\left(\frac{\|s_i^* - s^*\|}{h}\right)$. As discussed below and in Appendix A.1.1, this is a manifold-regularization or Laplacian-smoothing objective: it penalizes high-frequency variation across neighborhoods of expert states. Under the stated assumptions, that structure improves variance, stability, and generalization.

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.
- (ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.
- (iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^1$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

Thus, under the stated smoothness and graph assumptions, MRIL provides stronger generalization and rollout-stability guarantees than vanilla BC.

Proof sketch. Appendix A.1.1 gives the full proof; we summarize the argument here. The smoothness term is first written as a graph Laplacian penalty on a k -nearest-neighbor (k -NN) affinity graph with kernel weights w_i . In the large-sample limit, this penalty converges to a weighted Dirichlet energy (Belkin & Niyogi, 2008; Zhou et al., 2003). Minimizing that energy makes f locally Lipschitz almost everywhere, supporting smoothness and closed-loop stability, and also acts as Tikhonov regularization, reducing estimator variance while keeping bias controlled. $\square$

Intuitively, the regularizer uses the neighborhood graph as a data-dependent notion of where smoothness should matter. Rather than only penalizing pairwise disagreement, it reduces the local Lipschitz constant of f along high-density directions of the data distribution $p(s)$, so small state changes induce controlled action changes. This gives a structured form of interpolation on in-distribution (ID) states and local extrapolation to nearby out-of-distribution (OOD) states.

2.3 IMPLICIT MANIFOLD REGULARIZATION VIA IN-CONTEXT ARCHITECTURES

The explicit MRIL objective shows how local structure can improve over vanilla BC, but it also changes the learning problem. It introduces a smoothness weight λ that must be tuned and replaces the standard BC loss with a regularized objective that may alter optimization behavior. This motivates the architectural question: *can we obtain the same benefits conferred by MRIL (Eq. 1), by modifying the policy architecture rather than modifying the objective?*

We next introduce a retrieval-based architecture that induces an *implicit* manifold-regularization effect (iMRIL) while retaining the standard imitation objective. In iMRIL, the benefits of Laplacian smoothing from MRIL arise from the prediction architecture itself (Fig. 2), so no additional λ hyperparameter is needed. Section 2.4 then turns this construction into the practical method DARP.

iMRIL architecture: The key move in iMRIL is to place neighborhood aggregation inside the policy parameterization rather than inside the loss. Instead of training a feedforward predictor $f(s)$ with an explicit neighborhood-regularized objective (Eq. 1), we parameterize the action predictor $\hat{f}$ as an average over predictions at retrieved training states and train with the usual imitation objective. Concretely, iMRIL learns a per-state predictor f_θ such that the aggregated predictor $\hat{f}(s^*) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*)$, computed over nearest neighbor states $\{s_i^*\}_{i \in \mathcal{N}_k(s^*)}$ from the training set, matches the expert action a^* . With this parameterization, iMRIL optimizes at training time:

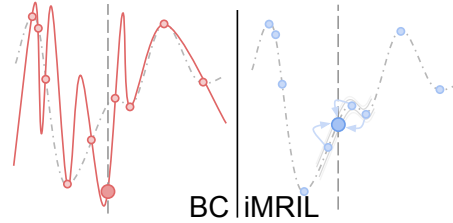


Figure 2: **iMRIL induces Laplacian-style smoothing through architecture**, reducing variance and enforcing local consistency across the neighborhood graph, while standard BC can fit jagged functions between supervised states when no comparable smoothness bias is present.

¹ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

$$\arg \min_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} \left[\left\| \underbrace{\left(\frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_{\theta}(s_i^*) \right)}_{\hat{f}(s^*)} - a^* \right\|_2 \right] \quad (2)$$

At deployment time, the same mechanism applies to any query state s_q : retrieve its k -NN from the training set and aggregate $\hat{a} = \hat{f}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_{\theta}(s_i^*)$. Section 2.4 shows that the form of f is not incidental; choosing a difference-aware parameterization is central to the empirical performance of iMRIL and motivates DARP.

This architecture learns f through a predictor $\hat{f}$ that is only observed after neighborhood aggregation over training states $f(s_i^*)$. Supervising the aggregated output discourages arbitrary non-smooth variation in the per-neighbor predictions, producing the same kind of local consistency described in Section 2.2. We formalize this connection by proving equivalence to Laplacian regularization in Section 2.2.

Equivalence between iMRIL and MRIL: We defer the full equivalence proof between MRIL and iMRIL to Appendix Section A.1, and state the main result with a proof sketch here.

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof sketch. Appendix Section A.1.2 contains the full derivation. The explicit regularizer can be solved spectrally by diagonalizing the k -NN Laplacian, which yields the eigenmode filter $(1 + \lambda \mu)^{-1}$. The implicit objective is neighbor aggregation $\hat{f} = S f$ with $S = D^{-1} A$, the random-walk matrix, which shares the relevant eigenvectors and applies the fixed filter $1 - \mu$. Both operations therefore act as graph low-pass filters: smooth modes with small eigenvalues are preserved, while rapidly varying, high-variance modes across neighbors are damped. Thus iMRIL performs Laplacian smoothing implicitly, reducing variance and enforcing local consistency without tuning λ . $\square$

The implicit-smoothing interpretation does not remove the need for a learned policy; it identifies how the architecture restricts the functions expressible after aggregation. The neighbor-conditioned network f_{θ} learns how expert actions change under local perturbations and proposes locally adapted actions for each neighbor. Aggregation then smooths those proposals across the retrieved neighborhood, so learning addresses local bias while aggregation controls variance and improves stability.

2.4 DIFFERENCE-AWARE RETRIEVAL POLICIES: A PRACTICAL INSTANTIATION OF iMRIL FOR IMITATION LEARNING

We now turn the iMRIL framework into a practical algorithm for imitation-learning problems at the scales considered in our experiments. Building on Eq. 2, we specify two design choices that make the method broadly usable with modern policy classes: the parameterization of the neighbor-conditioned predictor and the aggregation rule over retrieved neighbors.

2.4.1 DIFFERENCE-BASED PARAMETERIZATION OF f_θ

Eq. 2 leaves the parameterization and inputs of f_θ unspecified, making the architecture a natural place to encode a more transferable local representation. Because neighborhood aggregation should capture how expert actions change under local perturbations, f_θ should receive the *difference* between the query and each retrieved neighbor. In **Difference-Aware Retrieval Policies (DARP)**, we therefore move beyond the basic form $f_\theta(s_i^*)$ by also providing the neighbor action a_i^* and the displacement $\Delta s_i = s_i^* - s_q$. For query s_q and neighbor (s_i^*, a_i^*) , the predictor produces a candidate action $a'_i = f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)$.

Let $\mathcal{N}_k(s_q)$ be the index set of the k -nearest neighbors retrieved by a distance function $d(s_q, s_i^*)$.² Given this retrieved set, DARP predicts for any query s_q through the neighborhood aggregation described in Section 2.3:

$$\hat{a}_q = f_{\text{DARP}}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i \quad (3)$$

$$= \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q). \quad (4)$$

At training time, this can be used to define a straightforward imitation learning objective from the expert dataset $\mathcal{D}^*$:

$$\arg \min_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\|f_{\text{DARP}}(s_q^*) - a_q^*\|^2] \quad (5)$$

This objective optimizes the predictor parameters f_θ by matching the aggregated action $\hat{a}_q$ to the expert action a_q^* . Because the loss remains standard supervised imitation, f_θ can be implemented with common policy backbones; in our experiments we use feedforward or convolutional neural networks. Section 3 shows that the difference-based inputs are essential to performance. At inference time, DARP retrieves the k -NN for the query and applies the aggregation rule in Eq. 3.

2.4.2 GOING BEYOND LINEAR AGGREGATION

Averaging neighborhood predictions, $\hat{a}_q = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i$, is only one member of a broader family of permutation-invariant aggregation functions $g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q)})$. The aggregator can instead be parameterized by set-compatible neural architectures such as set transformers (Lee et al., 2019) or DeepSets (Zaheer et al., 2017), giving the generalized predictor from Eq. 3, $\hat{a}_q = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$. This broader view also allows DARP to model richer action distributions than the unimodal Gaussian implicit in the L_2 objective of Eq. 5. Instead of predicting $\hat{a}_q$ directly, DARP can predict distribution parameters α for $p(a_q; \alpha)$, such as Gaussian-mixture means, covariances, and weights, or the score function for a diffusion model (Pignat & Calinon, 2019; Chi et al., 2024). The same retrieval-and-aggregation structure can therefore be trained by maximum likelihood for multimodal action distributions:

$$\arg \max_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\log p(a_q^*; \alpha_\theta(s_q^*))], \text{ where } \alpha_\theta(s_q^*) = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q^*)\}_{i \in \mathcal{N}_k(s_q^*)}) \quad (6)$$

At test time, the policy retrieves neighbors for s_q , constructs $\alpha_\theta(s_q) = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$, and samples $\hat{a}_q \sim p(\cdot; \alpha_\theta(s_q))$ when the policy head is stochastic. Appendix Section A.3 provides detailed training pseudocode.

2.5 SUMMARY

In summary, local consistency in behavior cloning corresponds to manifold regularization through Laplacian smoothing and yields improvements in variance, smoothness, and stability under the assumptions of Theorem 1. Instead of adding an explicit regularizer and tuning λ , DARP builds

²In our work we use the Euclidean distance in a pre-trained embedding space, although other neighborhood functions are also applicable. We refer the reader to Appendix Section A.2.1 for a thorough discussion of design decisions in constructing neighborhood sets via retrieval.

neighborhood aggregation into the policy architecture, inducing the same smoothing effect implicitly (Theorem 2). The resulting algorithm remains in the standard imitation-learning data regime: for each expert pair (s_q^*, a_q^*) , DARP retrieves the k -nearest neighbor states from $\mathcal{D}^*$, forms difference vectors $\Delta s_i = s_i^* - s_q^*$, maps each tuple $(s_i^*, a_i^*, \Delta s_i)$ through f_θ to produce candidate actions, and aggregates them with a permutation-invariant function g_ψ . Training uses only the standard imitation objective.

3 EXPERIMENTAL EVALUATION

We evaluate DARP around three questions that probe the method across the benchmark settings supported by our study: **Q1**: Can DARP consistently outperform standard behavior cloning?, **Q2**: Can DARP handle more complex state representation and action distributions?, **Q3**: How do different architectural components contribute to DARP’s performance gains? The experiments span low-dimensional states, high-dimensional image features, and diverse action representations. The task suite covers continuous control (MuJoCo), simulated and real robotic manipulation (Robosuite, RoboCasa, FurnitureBench), and discontinuous environments designed to stress-test retrieval-based policies.

3.1 BASELINE COMPARISONS AND TASK DESCRIPTIONS

MuJoCo Tasks: The MuJoCo (Todorov et al., 2012; Fu et al., 2020) benchmarks evaluate locomotion control across several simulated bodies moving on a flat plane: Hopper (single-legged hopping robot), Walker (bipedal humanoid), Ant (quadruped), and HalfCheetah (biped).

Robosuite Tasks: The Robosuite (Zhu et al., 2020) manipulation tasks use a single robotic arm but require distinct object-interaction skills. Stack requires placing a smaller cube on a larger cube, Threading requires inserting a thin needle-like tool into a small ring, and Square Peg requires placing a square wooden block with a central hole onto a square peg.

RoboCasa Tasks: The RoboCasa (Nasiriany et al., 2024) benchmarks evaluate single-arm manipulation in randomized kitchen scenes. Drawer closes an open drawer, Door closes an open microwave door, and Stove turns a stove burner off by twisting a knob.

Baseline Comparisons: We compare DARP with baselines that represent nearest-neighbor retrieval, local regression, standard parametric learning, retrieval-conditioned transformers, and explicit smoothing: (1) **R&P (Sridhar et al., 2025)**: directly executes the action of the nearest neighbor, (2) **LWR (Pari et al., 2022)**: performs locally weighted regression over retrieved neighbors, (3) **BC**: is standard parametric behavior cloning, (4) **REGENT (Sridhar et al., 2025)**: is a transformer-based in-context method conditioned on retrieved neighbors, and (5) **MRIL**: is the explicitly smoothed version of DARP from Section 2.2.

3.2 CAN DARP CONSISTENTLY OUTPERFORM STANDARD BEHAVIOR CLONING? (Q1)

We first test the central hypothesis in low-dimensional state spaces, where state distances are directly interpretable and retrieval neighborhoods can be computed without learned visual embeddings. This evaluation includes MuJoCo locomotion and Robosuite/RoboCasa manipulation, with the robotic manipulation data generated by MimicGen (Mandlekar et al., 2023). In these experiments, the aggregation function g is the simple average of all neighbor action predictions a' .

Method	Hopper	Ant	Walker	HalfCheetah
R&P (Sridhar et al., 2025)	711.82 $\pm$ 85.63	-305.97 $\pm$ 76.42	419.18 $\pm$ 50.21	-178.64 $\pm$ 29.75
LWR (Pari et al., 2022)	1703.78 $\pm$ 245.95	846.59 $\pm$ 216.06	1484.91 $\pm$ 356.54	1945.82 $\pm$ 567.26
BC	2313.65 $\pm$ 203.75	2376.20 $\pm$ 339.43	2658.40 $\pm$ 274.08	1063.23 $\pm$ 371.08
REGENT (Sridhar et al., 2025)	1819.39 $\pm$ 186.24	-302.10 $\pm$ 146.67	507.01 $\pm$ 76.10	169.85 $\pm$ 63.10
MRIL	2793.63 $\pm$ 156.41	3869.08 $\pm$ 241.00	4370.96 $\pm$ 168.13	701.58 $\pm$ 195.08
DARP	3545.57 $\pm$ 3.54	4383.28 $\pm$ 266.37	4894.01 $\pm$ 75.12	5515.41 $\pm$ 841.33
DARP Set Transformer	2965.86 $\pm$ 103.08	4063.79 $\pm$ 218.80	4752.42 $\pm$ 109.23	3417.85 $\pm$ 764.57

Table 1: **DARP and DARP Set Transformer improve over the compared approaches on the evaluated MuJoCo tasks.** The table compares DARP, BC, and other baselines using low-dimensional state inputs. Scores are averaged across 100 independent trials with 95% confidence intervals. Parametric policies use Multi-Layer Perceptrons; diffusion-policy results are reported in Section 8.

Across the evaluated low-dimensional environments, DARP improves substantially over standard behavior cloning. Robotic manipulation gains range from 15-25 percentage points, and the locomotion benchmarks show large score increases as well (Table 1 and Table 2). Purely non-parametric retrieval methods (R&P and LWR) are less effective in these settings, while MRIL usually improves on vanilla BC but does not match the best scores obtained by the full DARP architecture.

Because the practical architecture in Section 2.4 is not tied to low-dimensional states, we next evaluate whether the same retrieval-and-difference framing extends to higher-dimensional visual representations.

3.3 CAN DARP HANDLE MORE COMPLEX STATE REPRESENTATION AND ACTION DISTRIBUTIONS? (Q2)

High-Dimensional Visual Input Representations. To test whether the method extends beyond compact state vectors, we evaluate DARP on simulated robotic manipulation tasks using R3M image embeddings (Nair et al., 2022). This setting asks whether retrieval remains useful when states are represented by high-dimensional

Method	Stack	Thrd.	Peg
BC	44	38	17
DARP	75	76	52

Table 3: Success rates (%) for vision-feature Robosuite tasks.



Figure 3: The “DrawerClose” RoboCasa task, where the robotic arm closes an open drawer in a kitchen scene.



Figure 4: The “Threading” Robosuite task, where the robotic arm inserts a needle-like implement into a small hole.

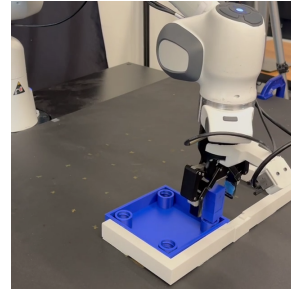


Figure 5: Real-world FurnitureBench square-table assembly, where the robot picks up a table leg and screws it into a tabletop corner.

	RoboCasa			Robosuite		Real	
Method	Drawer	Door	Stove	Stack	Thrd.	Peg	Sq. Table
BC	54	29	28	47	37	46	44
DARP	85	45	43	72	63	62	92

Table 2: Comparison of DARP and BC across the robotic evaluation environments using low-dimensional state features. Scores are success percentages (RoboCasa & Robosuite: 100 trials; Real: 50 trials). DARP improves over BC on every reported task and more than doubles BC’s real-world score. RoboCasa and Robosuite policies use Multi-Layer Perceptrons, while the real-world result uses diffusion policy; see Section 8.

visual feature vectors rather than simulator state (Table 3). DARP outperforms standard BC, with an average improvement of $\sim 35\%$, which is larger than the average low-dimensional Robosuite improvement ($\sim 22\%$). In these experiments, the difference-aware retrieval representation adapts to visual features more effectively than standard BC.

Multi-modal Action Distributions. We also evaluate a setting where action distributions are multimodal: on Push-T, DARP improves over behavior cloning by more than 20%. Appendix A.2.2 provides the full setup and results.

3.4 HOW DO DIFFERENT ARCHITECTURAL COMPONENTS CONTRIBUTE TO DARP’S PERFORMANCE GAINS? (Q3)

Ablation Study: To identify which architectural choices matter across the retrieval pipeline, we ablate the main components of DARP: (1) standard DARP; (2) DARP without neighbor actions; (3) an ensemble of 10 BC agents; (4) DARP with random neighbors instead of distance-based retrieval; (5) DARP using the L_2 norm of the distance vector; (6) the BC baseline using only the query state s_q ; (7) DARP using the query state rather than the query-neighbor difference vector; and (8) DARP with a permutation-dependent, rather than permutation-invariant, aggregator over the a ’s. Figure 6 reports the resulting ablation study.

The ablations indicate that the broadly useful components are the difference-vector representation and permutation-invariant aggregation, while neighbor actions contribute more modestly. Random retrieval degrades performance, showing that the neighborhood metric is an important part of the method. Replacing the permutation-invariant aggregator g with a permutation-dependent alternative also substantially hurts performance.

Divergence Analysis: To better understand why DARP can outperform standard BC in closed-loop execution, we analyze rollouts where BC fails but DARP succeeds. We define the “step of divergence” as the point at which the two policies begin receiving substantially different rewards. We set τ_r and τ_Δ to the 1st percentile of training-set likelihoods, so 1% of training deltas have likelihood below

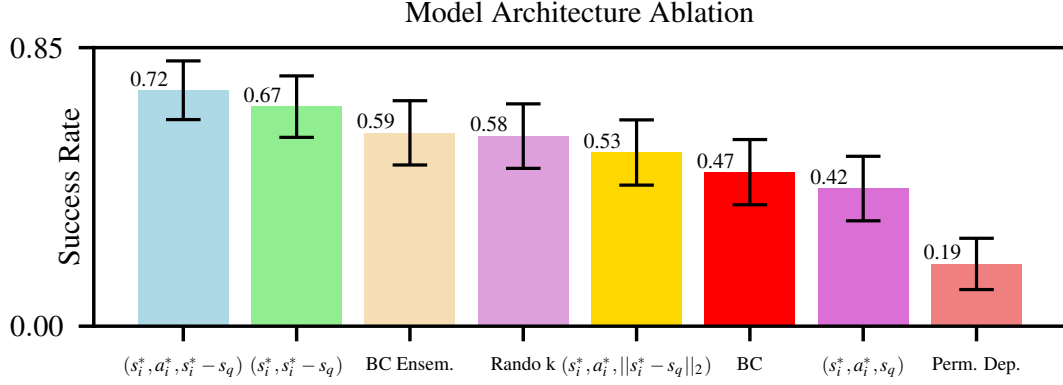


Figure 6: **Distance vectors and permutation invariance are central to DARP’s gains in this ablation.** Varying the core architecture shows that performance depends strongly on representing neighbor tuples as $(s_i^*, a_i^*, s_i^* - s_q)$ and aggregating them in a permutation-invariant way. Success rates are averaged across 100 Robosuite Stack trials with 95% confidence intervals.

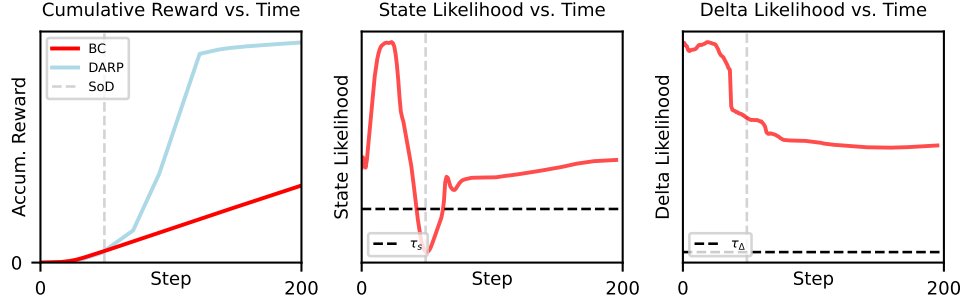


Figure 7: **Cumulative rewards for BC and DARP on Robosuite Stack show rollouts that initially match and then separate.** The vertical dashed line marks the step of divergence (“SoD”), after which BC fails and DARP succeeds. At the SoD, the state likelihood is $< \tau_s$ (OOD), while the delta likelihood is $> \tau_\Delta$ (in distribution).

τ_Δ . Across six rollouts from two tasks (Robosuite Stack and MuJoCo Hopper), the query state at the SoD has state likelihood $< \tau_s$ but delta likelihood $> \tau_\Delta$. This supports the hypothesis that DARP gains partly arise because the difference-vector parameterization remains in distribution for nearby deviations even when the absolute state is slightly out of distribution (Figure 7 shows one example with reward drift, SoDs, and likelihoods). Appendix A.2.4 provides additional robustness experiments.

4 RELATED WORK

Non-Parametric Imitation Learning Methods: Non-parametric and semi-parametric IL methods show that retaining access to local data structure can support generalization beyond a single fitted policy. VINN Pari et al. (2022) studies locally weighted regression for imitation and reports strong results in image embedding spaces, while MiDiGaP von Hartz et al. (2025) uses mixtures of Gaussian processes to model multimodal trajectories and generalize quickly. SEABO Lyu et al. (2024) uses retrieval for offline RL by rewarding transitions close to neighbors, thereby constructing a reward function. FlowRetrieval Lin et al. (2024), STRAP Memmel et al. (2025), and Behavior Retrieval Du et al. (2023) combine non-parametric retrieval with finetuning from large unlabeled datasets to improve generalization through test-time training. DARP shares the broader motivation of using retrieved experience, but differs by learning a semi-parametric policy over tuples $(s_i^*, a_i^*, s_i^* - s_q)$ rather than relying only on non-parametric aggregation or test-time adaptation. This gives the method a local variance-reduction mechanism while preserving the representational flexibility of parametric policies.

Smoothness-Constrained Policy Learning: A substantial body of work improves policy stability and robustness by imposing explicit smoothness constraints. L2C2 Kobayashi (2022) studies model-free RL under local Lipschitz constraints, obtaining smoothness and noise robustness without

removing expressiveness, and Asadi et al. (2018) develops a related Lipschitz-constrained approach for model-based RL. CCIL Ke et al. (2024a) extends this line to imitation learning by using a Lipschitz-constrained dynamics model to generate synthetic corrective labels. Similar smoothness motivations have also been scaled to humanoid controllers Chen et al. (2024) to reduce shakiness during deployment. DARP addresses the same broad stability concern, but induces smoothness through retrieval-conditioned architecture while keeping the training objective in the standard imitation-learning form.

In-Context Learning Methods: Recent work also studies retrieval from the perspective of in-context imitation learning. REGENT Sridhar et al. (2025) evaluates retrieval-augmented generalization by conditioning a causal transformer on retrieved states, actions, and rewards, while DPT Lee et al. (2023) uses supervised pretraining for transformers that predict actions from query states and in-context datasets, effectively learning how to explore. Other in-context architectures include ICRT Fu et al. (2024), Instant Policy Vosylius & Johns (2025), and KAT Di Palo & Johns (2024). These methods often emphasize adaptation to new tasks and environments; DARP instead targets improved performance and stability within the standard imitation-learning setting by reusing the demonstration dataset through a difference-aware retrieval policy.

5 CONCLUSION

We introduced **D**ifference-Aware **R**etrieval **P**olicies for Imitation Learning (**DARP**) (DARP), a nearest-neighbor-based policy class that reframes imitation learning around relative differences between query states and retrieved expert neighbors rather than only direct state-to-action mappings. The analysis shows that this architectural choice implicitly realizes Laplacian smoothing. Across the evaluated continuous-control and robotic-manipulation settings, the results support three conclusions within the paper’s experimental scope. First, DARP improves over standard behavior cloning with low-dimensional state representations. Second, the same retrieval-and-difference framing remains effective across the tested state representations, action-distribution modeling choices, and task complexities, with improvements ranging from 15-46%. Third, ablations show that difference vectors and permutation-invariant aggregation are important ingredients for the observed gains. Overall, the paper suggests that keeping demonstrations available as local references can broaden the practical robustness of behavior cloning without changing its supervision assumptions.

6 REPRODUCIBILITY STATEMENT

Supplementary source code, policy and environment configuration files, MuJoCo experiment data, scripts for generating Robosuite expert demonstrations with MimicGen, and modality-transformation code will be released upon acceptance. On NVIDIA L40 and L40s GPUs, deterministic runs are expected to reproduce the reported values. Results involving transformer components (REGENT and Set Transformer) may differ slightly from the reported numbers because those components are non-deterministic.

REFERENCES

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In *International Conference on Machine Learning*. PMLR, PMLR, 2018.
- Mikhail Belkin and Partha Niyogi. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.

-
- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies. *arxiv preprint arXiv:2410.11825*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. doi: 10.1090/cbms/092.
- Michael Jae-Yoon Chung, Maxwell Forbes, Maya Cakmak, and Rajesh PN Rao. Accelerating imitation learning through crowdsourcing. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4777–4784. IEEE, 2014.
- Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.
- Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha S. Srinivasa, and Abhishek Gupta. CCIL: continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Taisuke Kobayashi. L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4032–4039. IEEE, IEEE, October 2022.
- Jonathan Lee et al. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 43057–43083, 2023.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3744–3753, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.
- Li-Heng Lin, Yuchen Cui, Amber Xie, Tianyu Hua, and Dorsa Sadigh. Flowretrieval: Flow-guided data retrieval for few-shot imitation learning. In Pulkrit Agrawal, Oliver Kroemer, and Wolfram Burgard (eds.), *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pp. 4084–4099. PMLR, 2024. URL <https://proceedings.mlr.press/v270/lin25a.html>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR) 2023*, 2023. URL <https://arxiv.org/abs/2210.02747>. Originally appeared as arXiv preprint arXiv:2210.02747.

-
- Jiafei Lyu, Xiaoteng Ma, Le Wan, Runze Liu, Xiu Li, , and Zongqing Lu. Seabo: A simple search-based method for offline imitation learning. In *International Conference on Learning Representations*, 2024.
- Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireteyio Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.
- Elman Mansimov and Kyunghyun Cho. Simple nearest neighbor policy method for continuous control tasks. In *International Conference on Learning Representations (ICLR) 2018*, 2018. Under review as a conference paper at ICLR 2018.
- Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. Strap: Robot sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In Karen Liu, Dana Kulic, and Jeffrey Ichnowski (eds.), *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*, volume 205 of *Proceedings of Machine Learning Research*, pp. 892–909. PMLR, 2022. URL <https://proceedings.mlr.press/v205/nair23a.html>.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.
- Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. In *Robotics: Science and Systems (RSS)*. Robotics: Science and Systems Foundation, June 2022. doi: 10.15607/RSS.2022.XVIII-052.
- Emmanuel Pignat and Sylvain Calinon. Bayesian gaussian mixture model for robotic policy imitation. *IEEE Robotics and Automation Letters*, 2019. URL <https://arxiv.org/pdf/1904.10716.pdf>. Preprint version.
- Dean Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Comput.*, 3(1):88–97, 1991. doi: 10.1162/NECO.1991.3.1.88. URL <https://doi.org/10.1162/neco.1991.3.1.88>.
- Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 661–668. JMLR Workshop and Conference Proceedings, 2010.
- Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson, and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>.
- Steven L. Salzberg and David W. Aha. Learning to catch: Applying nearest neighbor algorithms to dynamic control tasks. In P. Cheeseman and R. W. Oldford (eds.), *Selecting Models from Data: Artificial Intelligence and Statistics IV*, volume 89 of *Lecture Notes in Statistics*, pp. 321–328. Springer, 1994. doi: 10.1007/978-1-4612-2660-4_33. URL https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33.
- Jonathan C. Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian D. Ziebart, and J. Andrew Bagnell. Feedback in imitation learning: The three regimes of covariate shift. *CoRR*, abs/2102.02872, 2021. URL <https://arxiv.org/abs/2102.02872>.
- Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. REGENT: A retrieval-augmented generalist agent that can act in-context in new environments. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.

-
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In Blai Bonet and Sven Koenig (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 3024–3030. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9590. URL <https://doi.org/10.1609/aaai.v29i1.9590>.
- Jan Ole von Hartz, Adrian Röfer, Joschka Boedecker, and Abhinav Valada. The unreasonable effectiveness of discrete-time gaussian process mixtures for robot policy learning. *arXiv preprint arXiv:2505.03296*, 2025.
- Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 30*, pp. 3391–3401. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/6931-deep-sets.pdf>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 16, 2003.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, Yifeng Zhu, and Kevin Lin. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

A APPENDIX

A.1 LEMMAS AND PROOFS

A.1.1 PROOF OF THEOREM 1

To prove Theorem 1, we begin from a standard graph-Laplacian characterization that applies to neighborhood-regularized learning on sampled data manifolds (Chung, 1997; Zhou et al., 2003; Belkin & Niyogi, 2008).

Lemma 1 (Smoothness regularizer as k -NN graph Laplacian penalty). *Let $\{s_1^*, \dots, s_n^*\}$ be the expert states with corresponding predicted actions $f(s_i^*) \in \mathbb{R}^{d_a}$. For each i , let $\mathcal{N}_k(s_i^*)$ denote the indices of the k -nearest neighbors of s_i^* (excluding i). Define asymmetric weights*

$$\tilde{W}_{ij} = \begin{cases} w_j(s_i^*), & \text{if } j \in \mathcal{N}_k(s_i^*), \\ 0, & \text{otherwise,} \end{cases}$$

and construct a symmetric affinity matrix

$$W_{ij} = \frac{1}{2} (\tilde{W}_{ij} + \tilde{W}_{ji}).$$

Let D be the degree matrix with $D_{ii} = \sum_j W_{ij}$, and define the k -NN graph Laplacian $L = D - W$.

Then the smoothness regularizer can be written as the quadratic form

$$\mathcal{L}_S(f) = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}_k(s_i^*)} w_j(s_i^*) \|f(s_i^*) - f(s_j^*)\|^2 \propto \text{Tr}(F^\top L F),$$

where $F = [f(s_1^*), f(s_2^*), \dots, f(s_n^*)]^\top \in \mathbb{R}^{n \times d_a}$. Equivalently, in the scalar case,

$$\mathcal{L}_S(f) \propto f^\top L f.$$

Corollary 1 (Continuum limit of smoothness regularizer). *Assume states $\{s_i^*\}_{i=1}^n$ are sampled i.i.d. from a smooth density $p(s^*)$ supported on an m -dimensional C^2 manifold $\mathcal{M} \subset \mathbb{R}^d$. Let W be the symmetrized k -NN affinity matrix constructed from a kernel K_Δ with bandwidth h , and let $L = D - W$ be the graph Laplacian.*

If $n \rightarrow \infty$, $h \rightarrow 0$, and $nh^{m+2} \rightarrow \infty$, then the normalized quadratic form converges to the weighted Dirichlet energy:

$$\frac{1}{n^2 h^{m+2}} \text{Tr}(F^\top L F) \longrightarrow C_K \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f(s)\|_2^2 p(s)^2 d\text{vol}(s),$$

where $C_K > 0$ is a constant depending only on the kernel K_Δ .

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.

(ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.

(iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^3$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

Thus, under the stated smoothness and graph assumptions, MRIL provides stronger generalization and rollout-stability guarantees than vanilla BC.

Proof. We establish the three claims in sequence.

(i) Variance reduction. The Laplacian penalty in $\mathcal{L}_{\text{MRIL}}$ is

$$\sum_{i,j} W_{ij} \|f(s_i^*) - f(s_j^*)\|^2 = 2f^\top L f,$$

where $L = D - W$ is the graph Laplacian and W is the k -NN affinity matrix. By Lemma 1, this is the empirical Dirichlet energy of f on the neighborhood graph. Standard results (Zhou et al., 2003; Belkin & Niyogi, 2008) identify this quadratic penalty with Tikhonov regularization in the graph Laplacian norm $\|f\|_L^2 = f^\top L f$. In statistical learning terms, the added Tikhonov penalty reduces estimator variance relative to the unregularized solution while keeping the bias controlled. Therefore, within this graph-regularized setting, MRIL has smaller estimator variance than vanilla BC, which has no comparable penalty.

(ii) Smoothness guarantee. Consider the continuum limit (Corollary 1): for i.i.d. samples $\{s_i^*\}$ from density p on a smooth manifold $\mathcal{M}$, the normalized penalty converges to

$$\int_{\mathcal{M}} \|\nabla f(s)\|^2 p(s)^2 d\text{vol}(s).$$

This quantity is the weighted Dirichlet energy of f on $\mathcal{M}$. When the integral is finite, f lies in the Sobolev space $H^1(\mathcal{M}, p^2)$ and is locally Lipschitz almost everywhere, with

$$\|f(s^*) - f(s'^*)\| \leq C \|s^* - s'^*\| \quad \text{for } p\text{-a.e. neighbor pairs } s^*, s'^*.$$

Therefore, minimizers of $\mathcal{L}_{\text{MRIL}}$ have bounded local Lipschitz constants along high-density regions of the state space. Vanilla BC has no comparable constraint: any oscillatory interpolant that matches the training data exactly attains the same supervised risk, so arbitrarily large Lipschitz constants remain possible.

(iii) Policy stability. Let $\Delta_t = \|s_t - s_t^*\|$ denote the deviation at time t . For Lipschitz dynamics T ,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|.$$

Decompose the action error:

$$\|f(s_t) - f^*(s_t^*)\| \leq \|f(s_t) - f^*(s_t)\| + \|f^*(s_t) - f^*(s_t^*)\|.$$

For vanilla BC, the first term $\|f(s_t) - f^*(s_t)\|$ is only minimized on the empirical distribution $P_{\mathcal{S}}$; off-distribution, it may be $O(1)$ regardless of Δ_t . The second term satisfies $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Hence the recursion can take the form

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(1) + O(\Delta_t)),$$

which accumulates linearly in t .

³ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

For MRIL, the Laplacian penalty enforces

$$\|f(s^*) - f(s'^*)\| \leq C\|s^* - s'^*\| \quad \text{for neighbor pairs } (s^*, s'^*),$$

as shown in part (ii). Thus $\|f(s_t) - f^*(s_t)\| = O(h^2)$ (where h is the radius of the kernel bandwidth around the point s_t) by local-linear regression error bounds, and $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Combining these,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a(O(h^2) + O(\Delta_t)).$$

Since the constant multiplying Δ_t is strictly smaller under the smoothness constraint, the cumulative error grows strictly slower than in vanilla BC. In particular, error growth is sublinear in the rollout horizon when h is small, whereas it is linear for vanilla BC.

Conclusion. Claims (i)–(iii) show that MRIL provides lower variance, local smoothness control, and slower rollout error accumulation than vanilla BC under the stated assumptions, completing the proof. $\square$

Kernel choice. For the neighborhood smoothness regularizer, we use a Gaussian kernel

$$w_i(s^*) \propto \exp\left(-\frac{\|s^* - s_i^*\|^2}{2h^2}\right), \quad \sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) = 1,$$

with bandwidth h set to the median distance to the k -th nearest neighbor across the dataset. This is a standard choice in manifold regularization (Belkin & Niyogi, 2008; Zhou et al., 2003) and gives the graph Laplacian penalty the usual continuum-limit connection to Dirichlet energy. Empirically, this default was stable across the evaluated tasks, and alternative kernels such as uniform k -NN or exponential decay produced qualitatively similar behavior.

A.1.2 PROOF OF THEOREM 2

We next state the lemmas needed to connect explicit Laplacian regularization and neighbor aggregation through their spectral forms.

Lemma 2 (Spectral form of explicit Laplacian regularization). *Let L be the symmetric normalized graph Laplacian with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$. The minimizer of the penalized objective*

$$\mathcal{L}_\lambda(f) = \|f - a^*\|^2 + \lambda f^\top L f$$

has the closed-form expansion

$$f_\lambda = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j.$$

Thus λ directly determines the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$ applied to each Laplacian mode.

Proof. Diagonalize $L = U \Lambda U^\top$ with $U = [u_1, \dots, u_n]$ orthogonal and $\Lambda = \text{diag}(\mu_1, \dots, \mu_n)$. Write $f = U c$, $a^* = U b$ in this basis. The objective becomes

$$\|U c - U b\|^2 + \lambda c^\top \Lambda c = \sum_{j=1}^n (c_j - b_j)^2 + \lambda \mu_j c_j^2.$$

Minimizing each term yields $c_j = \frac{1}{1 + \lambda \mu_j} b_j$. Transforming back to the original coordinates gives the stated expansion. $\square$

Lemma 3 (Spectral form of neighbor aggregation). *Let $S = D^{-1}A$ be the random-walk matrix of the k -NN graph, with adjacency A and degree D . For any prediction vector f , the neighbor-averaged prediction is $\hat{f} = S f$. In the Laplacian eigenbasis, this corresponds to the spectral filter*

$$\hat{f} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j,$$

i.e. $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Proof. By definition, $L = I - D^{-1/2}AD^{-1/2}$ and $S = D^{-1}A = I - L_{\text{rw}}$, where $L_{\text{rw}} = D^{-1}L$ is the random-walk Laplacian. Because L_{rw} and L share the same spectrum up to a similarity transform, the eigenbasis $\{u_j\}$ diagonalizes S . Hence each mode satisfies $Su_j = (1 - \mu_j)u_j$, which gives the claimed spectral filter. $\square$

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof. From Lemma 3, the neighbor aggregation operator $S = D^{-1}A$ acts on Laplacian eigenmodes u_j as

$$Su_j = (1 - \mu_j)u_j,$$

where μ_j are the normalized Laplacian eigenvalues. Thus, in the graph Fourier basis, neighbor aggregation multiplies each mode by the fixed spectral filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Lemma 2, in contrast, shows that explicit Laplacian regularization with parameter λ yields the spectral filter $\phi_\lambda(\mu) = (1 + \lambda\mu)^{-1}$. Both filters downweight high-frequency modes ($\mu \gg 0$) while preserving low-frequency modes ($\mu \approx 0$). The key difference is that $\phi_\lambda(\mu)$ requires tuning λ , whereas $\phi_{\text{DARP}}(\mu)$ is parameter-free.

To compare the filters locally, note that for small μ ,

$$\phi_{\text{DARP}}(\mu) = 1 - \mu \approx (1 + \mu)^{-1} = \phi_{\lambda=1}(\mu) \quad \text{up to } O(\mu^2) \text{ terms.}$$

Thus DARP can be interpreted as performing Laplacian smoothing with an effective regularization weight of order $\lambda \approx 1$ in normalized units. For large μ , $\phi_{\text{DARP}}(\mu)$ damps high-frequency modes strongly by driving them toward zero, yielding a sharper low-pass effect than the explicit regularizer in this comparison.

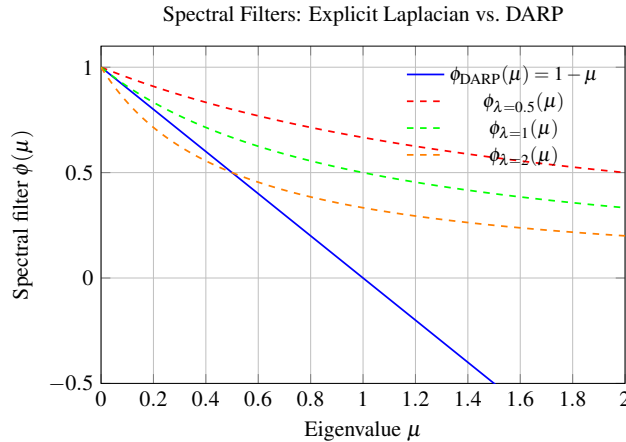


Figure 8: DARP induces a fixed low-pass spectral filter

Therefore, DARP’s aggregation step implements implicit Laplacian regularization with fixed spectral filter ϕ_{DARP} , eliminating the need to tune λ explicitly. $\square$

DARP can therefore be interpreted as *locally adaptive implicit regularization*: instead of introducing an explicit global weight λ , neighbor aggregation enforces smoothness through the graph structure. The effective regularization varies with local degree and neighborhood geometry, adapting to the density of the expert demonstrations. Spectrally, this is the fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$, which suppresses high-frequency modes more aggressively than the displayed explicit filters. Figure 8 illustrates this comparison and the resulting low-pass behavior without an additional smoothing hyperparameter.

A.2 ADDITIONAL EXPERIMENTAL DETAILS

A.2.1 RETRIEVAL

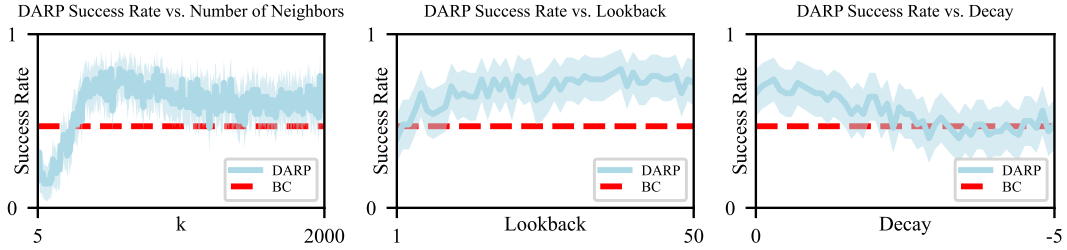


Figure 9: Retrieval-hyperparameter analysis for DARP on Robosuite Stack. Left: performance is low with very few neighbors, peaks around 500 retrieved neighbors, and plateaus slightly above BC as k approaches the dataset size. Center: incorporating more history generally helps, while little or no history can underperform BC. Right: performance depends on the decay applied to older observations; when decay is too high, behavior resembles little or no lookback and can fall below BC. Success rate is measured over 50 trials with 95% confidence intervals.

The distance function $d(s_q, s_i^*)$ determines which local evidence the policy can use, so retrieval quality is important. Simple Euclidean state distance can work, but our experiments use a modest extension that exploits the sequential nature of demonstrations by including history in the distance computation. Let the query trajectory be $S_q = (\dots, s_{q,-1}, s_{q,0})$, with $s_{q,0} = s_q$ the current query state. To compute $d(s_q, s_i^*)$ for an expert state s_i^* , we identify the source expert trajectory S_j^* and the index t of that state, so $s_i^* = s_{j,t}^*$. With lookback parameter ℓ , which specifies how many past states enter retrieval, the distance is:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\|$$

This sums Euclidean distances between the current and previous $\ell - 1$ states in the query and source trajectories, when the indices are valid. To emphasize recent observations during retrieval, we additionally use an exponential decay rate $\gamma \geq 0$:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\| \cdot e^{-\gamma n}$$

Figure 9 shows how success changes as these retrieval parameters vary in the evaluated environment.

A.2.2 CAN DARP HANDLE TASKS REQUIRING THE REPRESENTATION OF MULTI-MODAL ACTION DISTRIBUTIONS?

We evaluate DARP on Push-T (Chi et al., 2024) to test a setting where the policy must represent multimodal action distributions (Figure 10). In this experiment, DARP uses a Set Transformer head to predict Gaussian Mixture Model parameters. With this GMM head, DARP improves over BC by 22% on Push-T (Q2) (Table 4), showing that the retrieval architecture can be paired with multimodal action models.

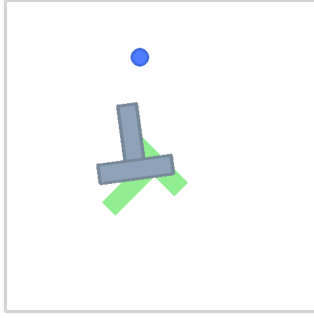


Figure 10: **Push-T Environment.** The policy controls the blue circle to push the T-shaped block.

Method	Score
BC	48 ± 8
DARP	70 ± 8

Table 4: **Push-T Results.** Across 100 trials, DARP improves over BC.

A.2.3 CAN DARP HANDLE DISCONTINUOUS ENVIRONMENTS WHERE NEARBY STATES MAY REQUIRE OPPOSING ACTIONS?

A natural concern for neighbor-based methods is discontinuity: nearby states under Euclidean distance may require very different actions. We therefore include a stress test based on a modified D4RL Umaze environment (Figure 11).

In this deliberately discontinuous environment, DARP reaches a 57% success rate compared with BC’s 25%. (Q3) (see Table 5) This result suggests that difference vectors and permutation-invariant aggregation can help distinguish useful from misleading neighbors, even when spatial proximity alone does not imply action similarity.

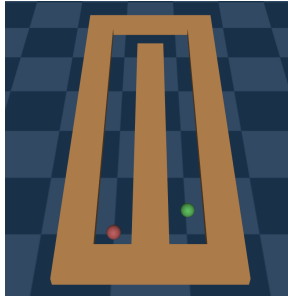


Figure 11: **Long maze environment.** The task moves a force-actuated ball from the green start region to the red destination.

Method	Succ. (%)
BC	25
DARP	57

Table 5: **Long maze results.** Over 100 trials, DARP improves over BC in the discontinuous maze setting.

A.2.4 CAN DARP RECOVER FROM BC ERROR?

To probe robustness to accumulated error, we roll out a BC agent in settings where it fails, fork the environment every k steps, and then roll out DARP from each cloned state. Figure 12 shows that DARP often recovers and achieves high scores even when BC has drifted toward failure and away from the expert support. This suggests that DARP is more robust to the slightly out-of-distribution states induced by BC’s accumulated errors.

A.2.5 COMPARISON WITH CCIL

Table 6 compares DARP with CCIL, which explicitly induces smoothness. Using the scores reported in the CCIL paper, we compare each method by percent improvement over BC. DARP has larger gains

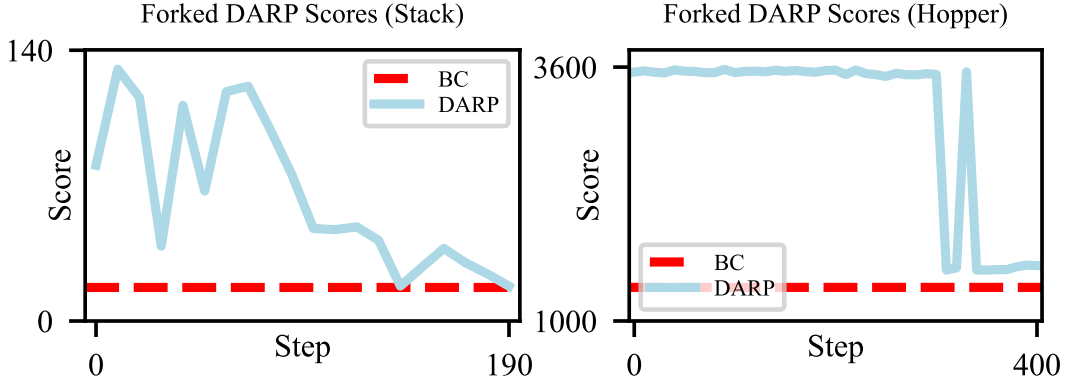


Figure 12: Forked-rollout analysis on Robosuite Stack and MuJoCo Hopper. We roll out a BC agent and fork the environment every k steps ($k = 10$ here), then start DARP from the forked state. DARP can still obtain high scores late in many failing BC rollouts, with recovery failing only around halfway through Stack and around 80% through Hopper.

Environment	Method	Improvement over BC (%)
Hopper	CCIL	32.6%
	DARP	53.2%
Walker	CCIL	84.1%
	DARP	84.1%
Ant	CCIL	12.6%
	DARP	84.5%
HalfCheetah	CCIL	5.4%
	DARP	418.7%

Table 6: **Relative improvement over BC compared to CCIL.** On the evaluated MuJoCo benchmarks, DARP achieves relative gains that are higher than or equal to CCIL.

on three of four environments and matches CCIL on Walker, with the largest gap on HalfCheetah (418.7% vs 5.4% improvement).

A.2.6 DISTANCE METRIC SENSITIVITY

Method	Success Rate (%)
BC	47
DARP (Cosine Similarity)	70
DARP (Euclidean Distance)	75

Table 7: **Distance Metric Sensitivity.** Success rates with R3M features on Robosuite Stack. Euclidean distance and cosine similarity give similar DARP performance, improving over BC by 28% and 23%, respectively.

Although the main experiments use Euclidean distance for retrieval, high-dimensional embeddings such as R3M make alternative metrics like cosine similarity natural. Table 7 shows that DARP with cosine similarity performs similarly to Euclidean retrieval, only 5% lower on this task. This suggests that the method is not overly sensitive to this metric choice in the evaluated visual-feature setting.

A.2.7 DARP IN COMBINATION WITH DIFFUSION POLICY

The main reported experiments use MLP backbones, but diffusion policy (Chi et al., 2024) is a strong imitation-learning model class for manipulation. Table 8 shows that DARP can be combined with

Method	Stack	Threading	Peg Insertion
BC (MLP)	34	12	28
DARP (MLP)	62	38	40
Diffusion	52	44	54
DARP w/ Diffusion	82	72	66

Table 8: **DARP in combination with diffusion policy.** DARP improves both MLP policies and diffusion policies on Robosuite Stack, Threading, and Peg Insertion. Scores are success rates across 50 state-based trials. This experiment uses fewer demonstrations than Table 2, so the BC and DARP numbers are not directly matched to that table.

diffusion rather than replacing it. With a diffusion backbone, DARP exceeds the MLP-backed version by 20%, 34%, and 26% success-rate points across the three tasks, and improves over standard BC by 48%, 60%, and 38%, respectively.

Method	Training (s/epoch)	Inference (s/step)
Diffusion	5.610	0.15700
DARP (MLP)	0.559	0.00437

Table 9: **Computational efficiency of DARP with an MLP backbone and diffusion policy.** Runtime comparison between DARP (MLP) and diffusion policy. In this setup, diffusion is $\approx 10\times$ slower during training and $\approx 36\times$ slower during inference.

We also find that DARP with an MLP backbone is substantially faster than standard diffusion in this setup, especially at inference time (Table 9).

A.2.8 DARP TRAINED ON HUMAN DEMONSTRATIONS

Method	Success Rate (%)
BC	45
DARP	60

Table 10: **Results with human demonstrations.** Success rates on Robosuite Stack when training from human demonstrations rather than RL-collected data.

The MuJoCo and Robosuite demonstrations used in the main experiments are collected by an optimal Reinforcement Learning policy. To check whether the gains extend to human-collected expert data, we also train on human demonstrations for Robosuite Stack. In that setting, DARP improves over standard BC by 15% (Table 10).

A.2.9 CHOICE OF RETRIEVAL HYPERPARAMETERS

Environment	Method	Score (Mean)
Hopper	BC	507.49
	DARP	805.69
Stack	BC	0.12
	DARP	0.31

Table 11: **Performance when validation loss selects epochs and retrieval hyperparameters.** Mean scores on Hopper and Stack show that DARP maintains a gain over BC under validation-based selection.

Figure 9 shows that poor retrieval choices, such as using very few neighbors, can make DARP underperform standard BC. Validation loss provides a practical way to select retrieval hyperparameters and training epochs that recover strong settings, as shown in Table 11.

A.3 PSEUDOCODE

Algorithm 1 gives pseudocode for the general DARP training loop.

Algorithm 1 Difference-Aware Retrieval Policies

```

1: Input: Expert demonstrations  $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ , number of neighbors  $k$ 
2: Initialize:  $f$  parameters  $\theta$ 
3: if  $g$  is parametric then
4:   Initialize:  $g$  parameters  $\psi$ 
5: end if
6: // Training Loop
7: while not converged do
8:   Sample batch of query data  $(s_q^*, a_q^*) \sim \mathcal{D}^*$ 
9:   for each query pair  $(s_q^*, a_q^*)$  in batch do
10:    // Find  $k$ -Nearest Neighbors from the entire dataset  $\mathcal{D}^*$ 
11:     $\mathcal{N}_k(s_q^*) \leftarrow \arg \min_{-k; d(s_q^*, s_j^*)}$ 
12:    // Compute Neighbor-based Predictions
13:    for each neighbor index  $i \in \mathcal{N}_k(s_q^*)$  do
14:       $a'_i \leftarrow f_\theta(s_i^*, a_i^*, s_i^* - s_q^*)$ 
15:    end for
16:    // Aggregate Predictions
17:    if  $g$  is parametric then
18:       $\hat{a}_q \leftarrow g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
19:    else
20:       $\hat{a}_q \leftarrow g(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
21:    end if
22:  end for
23:  // Update Parameters based on the batch loss
24:   $\mathcal{L} \leftarrow \sum_{(s_q^*, a_q^*) \in \text{batch}} \|\hat{a}_q - a_q^*\|^2$ 
25:  // Gradient descent step
26:   $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
27:  if  $g$  is parametric then
28:     $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$ 
29:  end if
30: end while
31: Output: Trained parameters  $\theta$  and, if applicable,  $\psi$ 

```

A.4 RUNTIME ANALYSIS

Environment	k	Train (s/epoch)	Test (s/step)
Hopper (1,000 datapoints)	BC	0.102	0.00127
	100	0.126	0.00413
	200	0.128	0.00418
	300	0.130	0.00420
	400	0.131	0.00421
	500	0.130	0.00419
Stack (4,200 datapoints)	BC	0.431	0.00139
	100	0.556	0.00437
	250	0.539	0.00442
	500	0.559	0.00437
	750	0.576	0.00433
	1000	0.591	0.00478
	1500	0.682	0.00452
	2000	0.758	0.00451

Table 12: **Runtime comparison across environments.** Training and inference times (seconds) for Behavior Cloning (BC) and multiple values of k on Hopper and Stack datasets.

Table 12 reports the training-time and inference-time overhead of DARP. In these measurements, cost grows sub-linearly with k , and inference remains practical for real-time robotic control. For example, on Robosuite Stack with $k = 500$, inference takes approximately 0.00437 seconds per step on our hardware, corresponding to a control frequency above 230 Hz.